



COURS DJANGO – IFNTI SOKODÉ

Mise en route

D'un terminal vide à une application qui tourne.

DEUX MINUTES, AVANT TOUT LE RESTE

**Confieriez-vous une calculatrice à
quelqu'un qui ne sait pas *compter*?**

L'IA est autorisée, et même attendue

Ce qui ne prouve plus rien

- Du code Django correct
- Un TP qui tourne le soir même
- Un dépôt bien rempli

DONC

Ce qui est évalué

- Quiz papier de 15 min, sans machine
- Le projet que vous rendez
- Soutenance de 10 min sur votre historique git

LE COROLLAIRE

**En cours et en TP, rien n'est noté.
Vous avez le droit de vous *tromper*.**

Au programme

01 Pourquoi Django

ce que le framework apporte, et ce que vous connaissez déjà

03 Anatomie d'un projet

projet, applications, et le trajet d'une requête

05 L'admin et la première page

saisir des données, puis les afficher

02 L'outillage

uv : Python, environnement et dépendances en un outil

04 Modèles et migrations

décrire une table en Python, la créer en base

06 Le fil rouge

le projet du semestre : le suivi des stages



01

Pourquoi Django

Qu'est-ce que Django ?

Un framework web Python qui fournit **déjà écrit** presque tout ce qui n'est pas votre métier.

- Une couche base de données — l'ORM, et ses migrations
- Une interface d'administration engendrée automatiquement
- Authentification, permissions, sécurité web
- Un moteur de templates, un routage, un serveur de développement

VOUS N'ARRIVEZ PAS LES MAINS VIDES

Ce que Laravel vous a déjà appris

RÔLE	LARAVEL	DJANGO
La commande qui fait tout	php artisan	manage.py
Décrire les données	Eloquent	l'ORM Django
Faire évoluer la base	migrations	migrations
Les pages	Blade	templates Django
Le routage	routes/web.php	urls.py
Les dépendances	composer	uv

LE CONTRAT

Django écrit l'administration, l'ORM, les migrations, les comptes. Vous écrivez le métier.



02

L'outillage

UN SEUL OUTIL



```
● ● ● bash

$ curl -LsSf https://astral.sh/uv/install.sh | sh
$ uv init --no-package suivi_stages && cd suivi_stages
Initialized project `suivi-stages` at `~/suivi_stages`
$ uv python pin 3.14
Pinned `.python-version` to `3.14`
$ uv add django==5.2.*
+ django==5.2.17
```

⚠ uv n'est pas dans les dépôts Ubuntu : ni `apt install uv`, ni `pip install uv`. On passe par l'installateur officiel, une seule fois par machine.

3.14

POURQUOI ÉPILER

la *même* version de Python sur toutes les machines de la salle — donc les mêmes messages d'erreur.

Les cinq commandes uv du semestre

uv python pin 3.14 fixe la version de Python du projet

uv add django ajoute une dépendance et l'installe

uv sync reconstruit l'environnement à l'identique, après un clone

uv run exécute dans l'environnement du projet

uv run manage.py ... toutes les commandes Django passent par là



03

03

Anatomie d'un projet

Le projet, puis l'application

● ● ● bash

```
$ uv run django-admin startproject suivi_stages .  
$ uv run manage.py startapp stages  
$ uv run manage.py runserver  
Starting development server at http://127.0.0.1:8000/
```



Le point final de la première commande n'est pas décoratif : sans lui, Django crée un dossier supplémentaire et vous vous retrouvez avec `suivi_stages/suivi_stages/` .

Deux dossiers, deux rôles

```
├─ manage.py
├─ suivi_stages
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└─ stages
    ├── models.py
    ├── views.py
    ├── admin.py
    └── migrations/
```

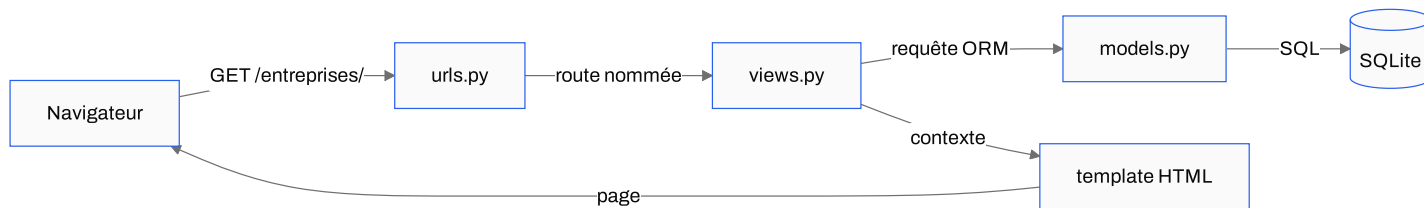
Le projet — suivi_stages/

La configuration : réglages, routage racine, point d'entrée du serveur. Il y en a un seul.

L'application — stages/

Un morceau fonctionnel : ses modèles, ses vues, son administration. Il y en aura plusieurs.

Qui appelle qui



Le template ne parle jamais à la base. La vue est le seul chef d'orchestre.

Trois lignes de settings.py qui comptent

```
INSTALLED_APPS = [  
    "django.contrib.admin",  
    "django.contrib.auth",  
    "django.contrib.contenttypes",  
    "django.contrib.sessions",  
    "django.contrib.messages",  
    "django.contrib.staticfiles",  
    "stages",  
]  
  
DEBUG = True
```

- 1 **INSTALLED_APPS** — Django ne connaît que les applications déclarées ici. Votre application ajoutée par `startapp` n'y est pas : c'est à vous de l'inscrire.
- 2 **Les quatre premières** — l'admin, les comptes, les permissions, les sessions. Vous ne les avez pas écrites, elles fonctionnent déjà.
- 3 **DEBUG** — les pages d'erreur bavardes que vous allez adorer aujourd'hui. Elles seront à couper le jour de la mise en ligne.



04

Modèles et migrations

Une classe Python, une table

```
from django.db import models

class Entreprise(models.Model):
    nom = models.CharField(max_length=120, unique=True)
    ville = models.CharField(max_length=80)
    secteur = models.CharField(max_length=80)
    contact = models.EmailField()

    class Meta:
        ordering = ["nom"]

    def __str__(self):
        return f"{self.nom} ({self.ville})"
```

Du modèle à la table



makemigrations

Django compare vos modèles
à l'état précédent et écrit un
fichier de migration



sqlmigrate

affiche le SQL que cette
migration exécutera — sans
rien appliquer



migrate

applique à la base ce qui ne
l'est pas encore

À RETENIR

**Le modèle
est la source
de vérité. La
base *suit*.**



05

L'admin, puis la première page

L'administration, gratuitement

```
from django.contrib import admin

from .models import Entreprise

@admin.register(Entreprise)
class EntrepriseAdmin(admin.ModelAdmin):
    list_display = ["nom", "ville", "secteur"]
    search_fields = ["nom", "ville"]
```

L'admin est sur <http://localhost:8000/admin/>, derrière un compte à créer :

```
uv run manage.py createsuperuser
```

- 1 **register** — sans cette déclaration, votre modèle n'apparaît pas dans l'admin. Django n'affiche que ce qu'on lui déclare.
- 2 **list_display** — les colonnes de la page de liste. Par défaut, il n'y en a qu'une.
- 3 **search_fields** — une barre de recherche apparaît, qui cherche dans ces champs-là.

Trois pièces, jamais une de moins

La route

- `stages/urls.py`, avec `app_name`
- incluse dans `urls.py` du projet
- on nomme chaque route

La vue

- interroge les modèles
- choisit le template
- lui passe un contexte

Le template

- `stages/templates/stages/`
- affiche, ne calcule pas
- l'emplacement suit une convention

Deux fichiers, une seule adresse

```
# stages/urls.py
from django.urls import path
from . import views

app_name = "stages"
urlpatterns = [
    path("entreprises/", views.liste_entreprises,
        name="liste_entreprises"),
]

# suivi_stages/urls.py
urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("stages.urls")),
]
```

- 1 **app_name** — l'espace de noms de l'application. C'est lui qui permettra d'écrire `{% url 'stages:liste_entreprises' %}` sans ambiguïté.
- 2 **name** — on nomme la route, et on ne réécrit jamais l'URL en dur ailleurs. Le jour où `/entreprises/` devient `/partenaires/`, un seul fichier change.
- 3 **include** — le projet ne connaît pas les routes de l'application : il lui délègue tout un préfixe. Ici la chaîne vide, donc la racine.

Une fonction qui reçoit et qui rend

```
from django.shortcuts import render

from .models import Entreprise

def liste_entreprises(request):
    return render(
        request,
        "stages/liste_entreprises.html",
        {"entreprises": Entreprise.objects.all()},
    )
```

- 1 **request** — toute vue reçoit la requête en premier argument, toujours. Elle porte l'utilisateur connecté, les paramètres, la méthode HTTP.
- 2 **La requête ORM** — `.all()` ne va pas encore chercher les données : la requête ne partira qu'au moment de l'affichage. On y reviendra.
- 3 **render** — trois arguments : la requête, le chemin du template, et le contexte. Le contexte est un dictionnaire : ses clés deviennent les variables du template.

Un chemin qui se répète

```
└─ stages
   └─ templates
      └─ stages
         └─ liste_entreprises.html
      └─ views.py
      └─ urls.py
```

Le dossier `templates/` **n'existe pas.** `startapp` ne le crée pas : c'est à vous.

Le sous-dossier au nom de l'application non plus, et c'est lui qui surprend.

Django cherche les templates de *toutes* les applications dans un seul espace. Sans ce sous-dossier, deux applications ayant chacune un `liste.html` entreraient en collision — et la première trouvée gagnerait.

Deux syntaxes, et c'est tout

```
<h1>Entreprises partenaires</h1>

<ul>
  {% for entreprise in entreprises %}
    <li>{{ entreprise.nom }} – {{ entreprise.ville }}</li>
  {% empty %}
    <li>Aucune entreprise enregistrée pour le moment.</li>
  {% endfor %}
</ul>
```

- 1 **Les accolades doubles** affichent une variable du contexte. `entreprise.nom` suit l'objet, sans parenthèses même pour un appel de méthode.
- 2 **Les accolades à pourcent** pilotent : boucles, conditions, héritage de templates. Ce sont les balises.
- 3 **empty** — le cas de la liste vide, donné par le langage. Aucune raison d'écrire un `if` autour de la boucle.



06

Le fil rouge

Le suivi des stages de l'IFNTI

Les entreprises publient des **offres**, les étudiants **candidatent**, les stages retenus donnent lieu à une **convention** et à une **évaluation**.

Entreprise

Offre

Etudiant

Candidature

Stage

Convention

Evaluation



Le dépôt du cours porte un **tag par point de reprise** : tp2-depart , tp2-corrige , et ainsi de suite. Une séance manquée ne bloque donc personne — on repart du tag du jour.

MAINTENANT

Ce que vous allez faire pendant deux heures et demie

- Installer `uv`, créer le projet, épingler Python **3.14**
- Créer l'application, la déclarer, **diagnostiquer deux pannes** provoquées exprès
- Écrire le modèle `Entreprise` — la spécification est **incomplète**, à vous d'arbitrer
- Saisir cinq entreprises réelles dans l'admin
- Afficher la liste sur `/entreprises/`
- Committer, pousser, déposer l'adresse sur Moodle



Ne restez jamais bloqué plus de **dix minutes** sans appeler. Je passe voir chacun deux fois.

C'est en forgeant que l'on devient forgeron

Ce cours s'oublie, ce que vous codez reste

L'ÉNONCÉ EST SUR MOODLE – PROCHAINE SÉANCE : MODÉLISER LE DOMAINE